具身智能VLA模型在昇腾平台的适配优化关键技术实践

分享嘉宾：翟安邦(CANN技术专家)

2026.4.18　中国·北京

content

目录

01 Pi0具身智能机器人VLA大模型简介

02 昇腾具身AI平台基础软硬件架构简介

03 Pi0昇腾A2平台推理适配方案及性能优化分析

04 Pi0昇腾推理优化策略及优化结果

05 Pi0模型昇腾真机部署实验方案

06 Pi0在昇腾的训推加速进一步优化方向

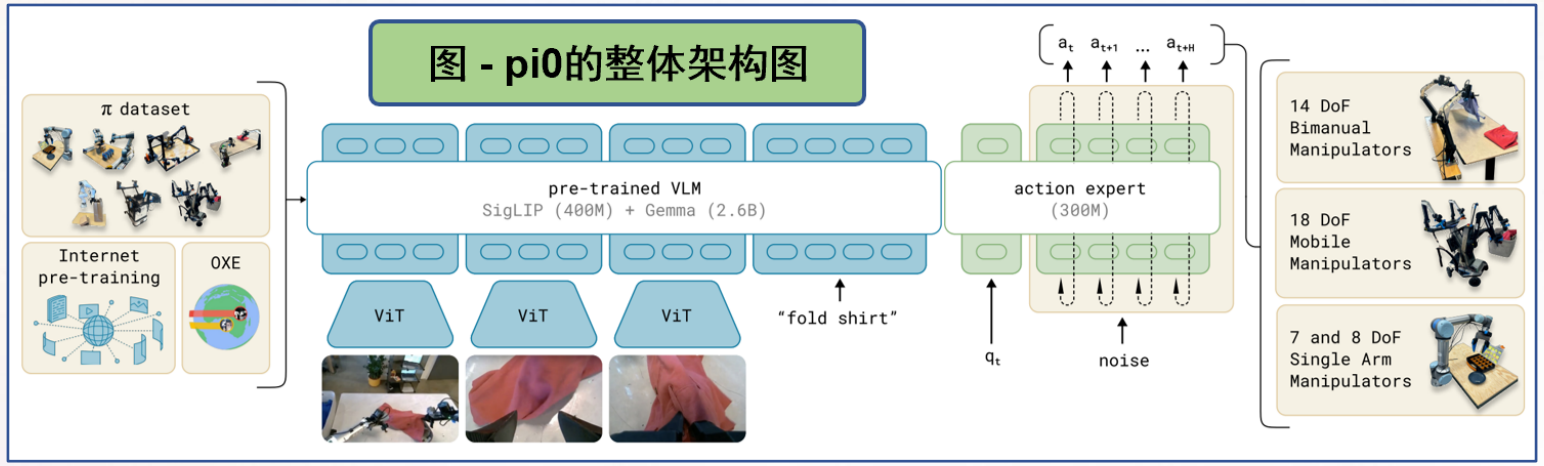
07 cann-recipes昇腾具身智能开源仓介绍

Pi0具身智能机器人VLA大模型简介

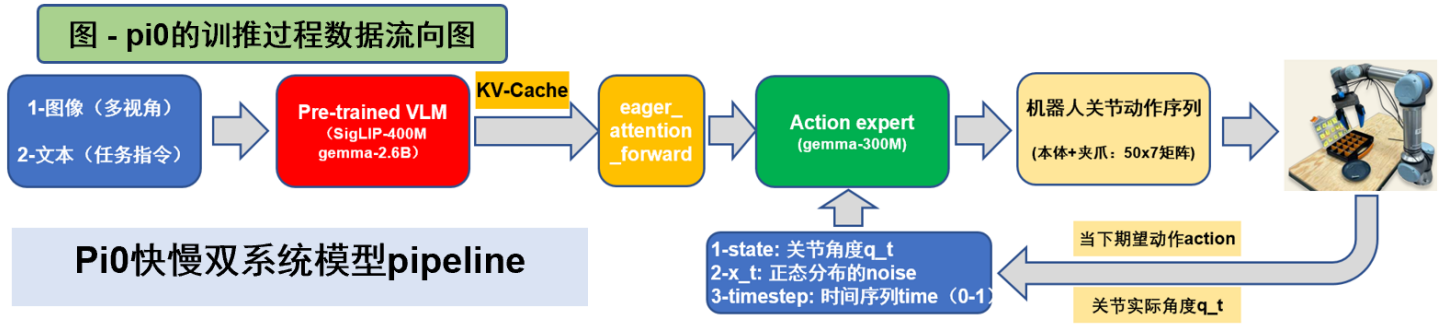
01

01 Pi0具身智能机器人VLA大模型简介

➢ Pi0是什么模型，有什么功能，为什么选择Pi0做昇腾适配优化



模型拆解



Pi0快慢双系统模型pipeline

Pi0真机控制操作视频

➢ Pi0是机器人VLA端到端操作大模型，模型大小为3.3B，结合了预训练的VLM多模态大模型(3B）和动作专家DIT小模型(0.3B)

➢ 和传统的VLA模型相比，利用diffusion transformer架构，有助于生成高维的连续动作值输出场景

➢ 在真实机器人操作效果上，泛化能力超越传统openVLA、ACT等模型，在叠衣服、桌面清理、装盒等复杂任务上表现优异。同时，在5-20分钟的长时序多阶段任务中，展现出强大的鲁棒性和泛化能力。

昇腾具身AI平台基础软硬件架构简介

02

02 昇腾具身AI平台基础软硬件架构简介

➢ 昇腾生态做具身智能有哪些软硬件可以用

行业场景化应用

视图分析、检索聚类、搜索推荐、自然语言处理、

智能语音、机器人具身智能、自动驾驶.....

昇腾AI基础软硬件架构

ModelArts应用使能

训练平台 / 推理引擎 / 模型套件 / SDK

AI计算框架

昇思MindSpore

PyTorch / TensorFlow / Caffe.....

异构计算架构

CANN

多样化昇腾处理器形态

模组 / 小站 / 加速卡 / 服务器 / 集群

昇腾赋能千行百业

具身 / 大模型 / 金融

具身大脑

视觉大模型场景理解

VLA大模型

高级子任务/机器人动作

实时图像 / 机器人状态

小脑 + 机器人

机械臂执行动作

夹爪执行抓取

昇腾系列服务器

•CANN + PyTorch-NPU

•在线一行语句自动迁移

•使能大模型并行计算加速

VLA端到端大模型

多模态输入

场景理解

任务规划

机器人精准运动

人形机器人

端侧AI智慧视觉

Atlas 800I A2 / OrangePi AI Studio / OrangePi AIpro等

云边端协同 - 具身智能算力解决方案

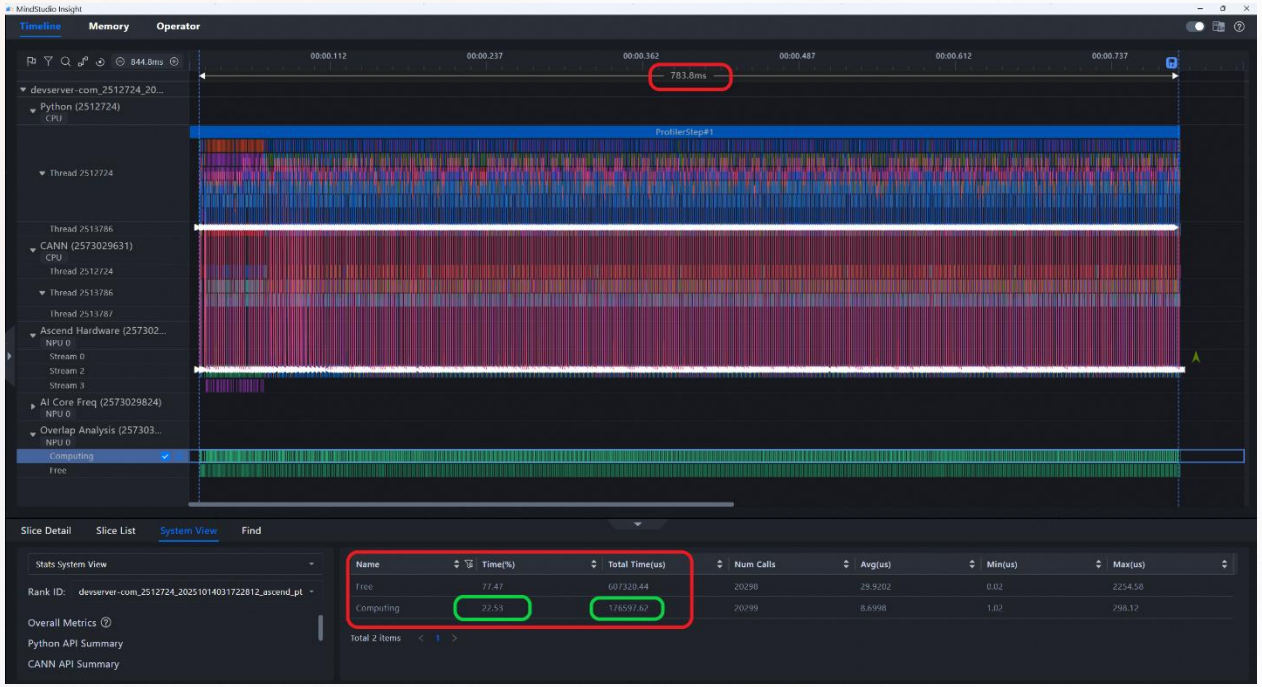
全面使能AI开发者，共享行业数字未来

Pi0昇腾A2平台推理适配方案及性能优化分析

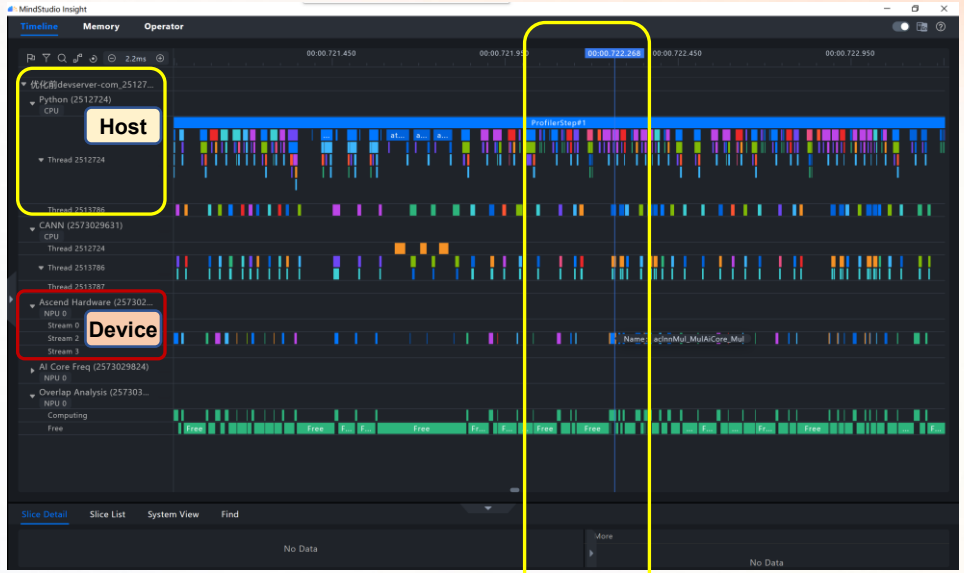
03

03 Pi0在昇腾上的开箱适配效果分析

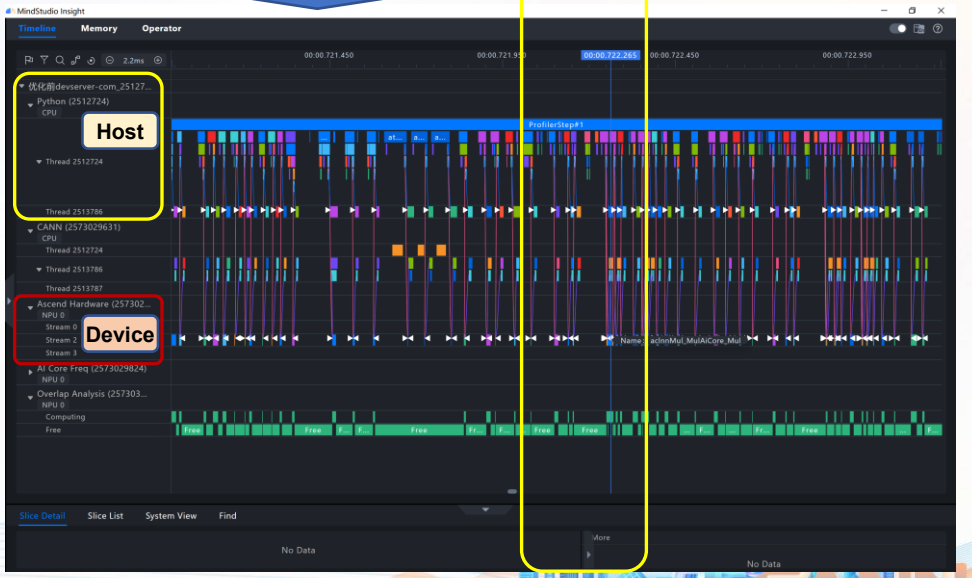
➢ Pi0在昇腾A2上如何适配跑通，如何分析流水图，如何发现pi0推理性能提升点



Pi0昇腾开箱适配推理性能流水图(未优化性能)



显示流水



Pi0昇腾开箱推理流水时间轴放大图

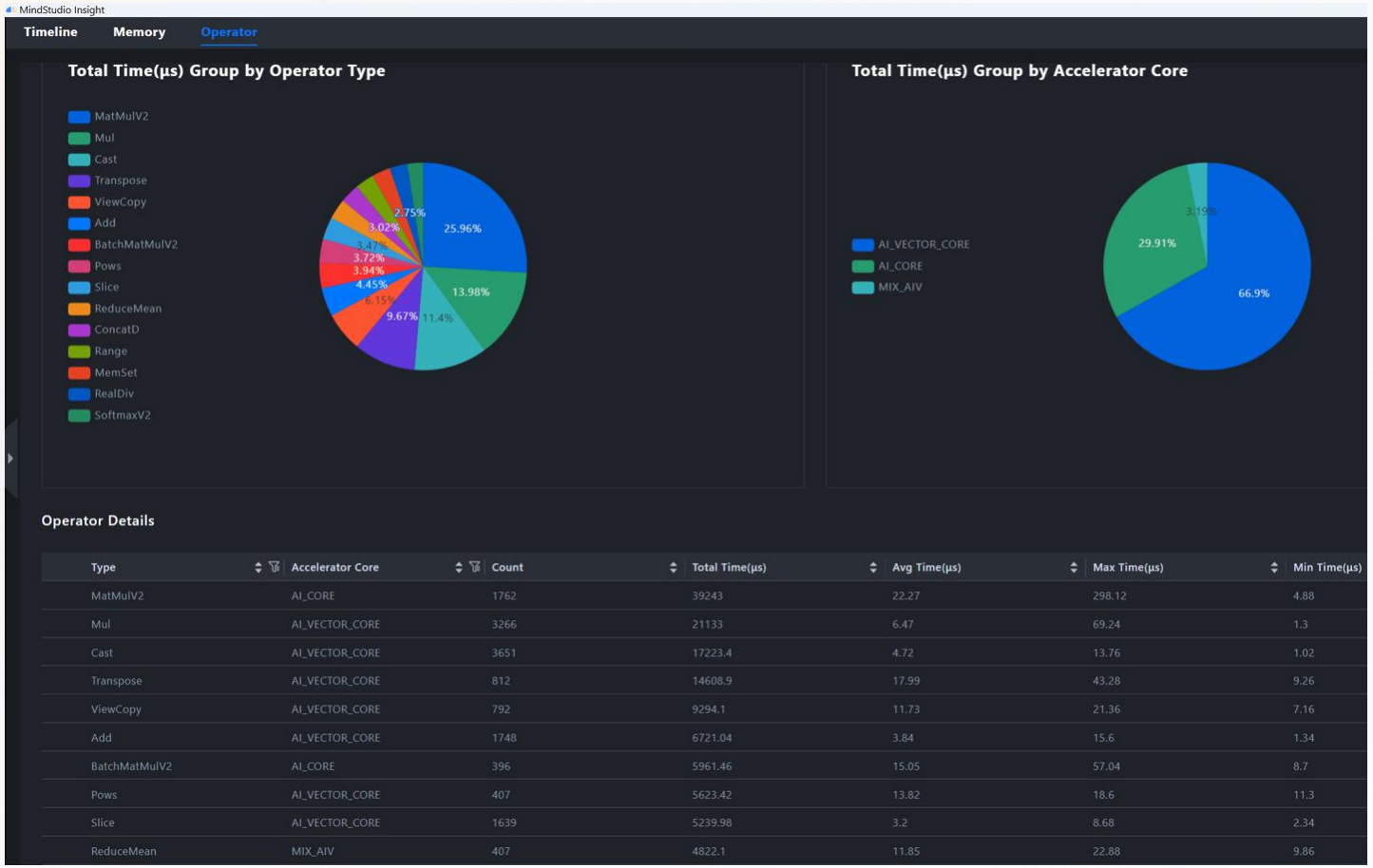
（1）昇腾适配：一行语句自动适配迁移，即可正常推理

```
1. import torch_npu
```

（2）通过流水图进行优化点分析：从上图中可以看到，pi0在开箱未优化推理过程中，cpu侧到npu侧的指令下发次数很多，且npu空闲时间占比较大，是典型的host-bound问题，后续可以进一步通过图模式、计算逻辑优化等策略减少指令下发次数，从而将昇腾npu计算时间总占比从22.5%提升至94%，提升昇腾NPU计算单元的有效利用率。

03 AI_CORE算子占比分析及推理性能优化思路

➢ Pi0在昇腾A2推理中如何分析算子性能，如何进一步发现pi0推理性能提升点



Type	Accelerator Core	Count	Total Time(μs)	Avg Time(μs)	Max Time(μs)	Min Time(μs)
MatMulV2	AI_CORE	1762	39243	22.27	298.12	4.88
Mul	AI_VECTOR_CORE	3266	21133	6.47	69.24	1.3
Cast	AI_VECTOR_CORE	3651	17223.4	4.72	13.76	1.02
Transpose	AI_VECTOR_CORE	812	14608.9	17.99	43.28	9.26
ViewCopy	AI_VECTOR_CORE	792	9294.1	11.73	21.36	7.16
Add	AI_VECTOR_CORE	1748	6721.04	3.84	15.6	1.34
BatchMatMulV2	AI_CORE	396	5961.46	15.05	57.04	8.7
Pows	AI_VECTOR_CORE	407	5623.42	13.82	18.6	11.3
Slice	AI_VECTOR_CORE	1639	5239.98	3.2	8.68	2.34
ReduceMean	MIX_AIV	407	4822.1	11.85	22.88	9.86

Pi0昇腾开箱推理算子计算时间占比统计图(未优化性能)

➢ **通过参与计算的算子类型占比进行性能分析：**需关注推理过程中AI_VECTOR_CORE和AI_CORE中算子所占的比例。pi0原始开箱推理性能AI_CORE大算力单元算子占比从29.9%，经过昇腾npu亲和的融合大算子替换优化，AI_CORE大算力单元算子占比提升至50%。充分发挥昇腾NPU的Cube(大算力矩阵运算单元）的计算性能。

推理性能优化思路

➢ 需要着重减少pi0推理过程中**CPU到NPU的指令下发次数，尽量使用昇腾亲和融合大算子**。

➢ 需推动计算逻辑下沉至NPU侧，通过整图编译优化消除零散算子的调度开销，并针对**旋转编码、Flash Attention、LayerNorm、QKV计算等关键性能瓶颈模块**，逐一替换为昇腾原生亲和算子，**最大化发挥AI_CORE的Cube矩阵计算能力**。

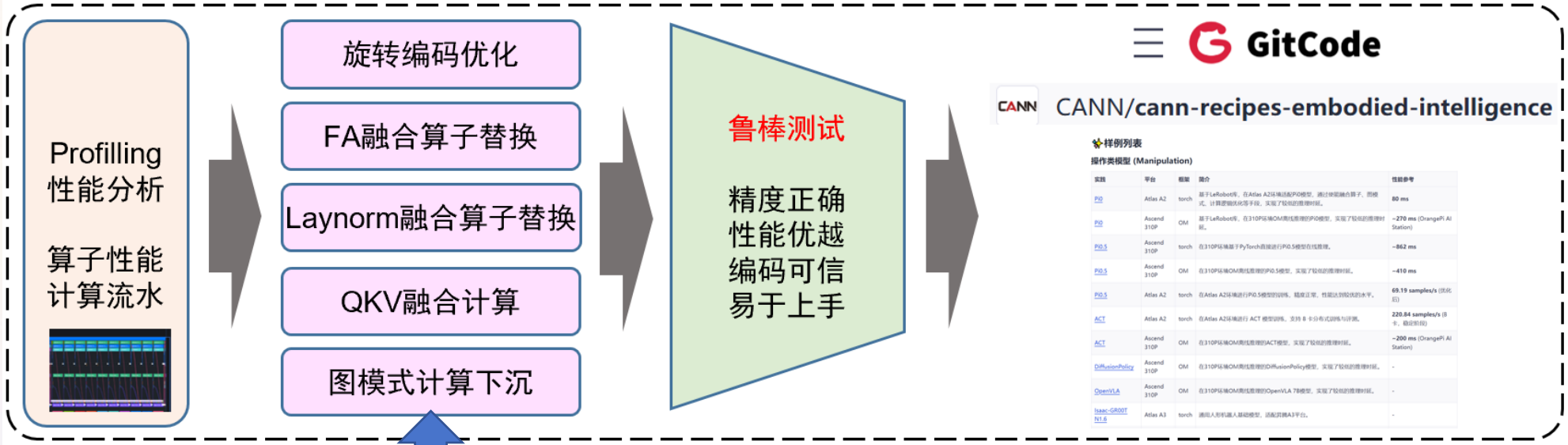
➢ 需优化CPU与NPU间的数据传输链路，减少冗余数据交互，从“指令调度”“算子适配”“数据传输”等维度形成协同优化方案，**解决host-bound问题与算力碎片化痛点，实现推理性能的质效提升**。

Pi0昇腾推理优化策略及优化结果

04

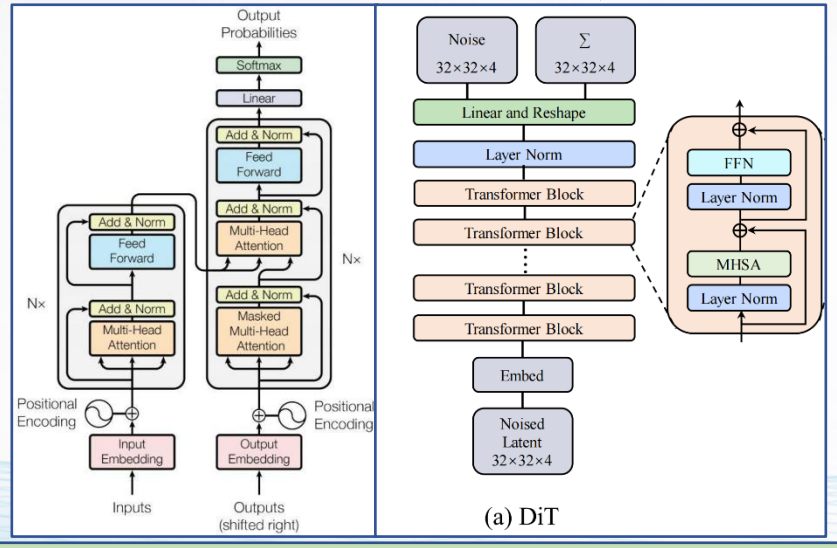
04 Pi0昇腾推理优化策略及优化结果

➢ Pi0在昇腾A2推理有哪些优化手段，能把推理性能从简单跑通提升至多快



Pi0昇腾适配-性能优化-开源开放整体流程图

➢ 融合算子替换
➢ 计算逻辑优化



Pi0机器人大模型基础架构算子模块图：Transformers + DiT

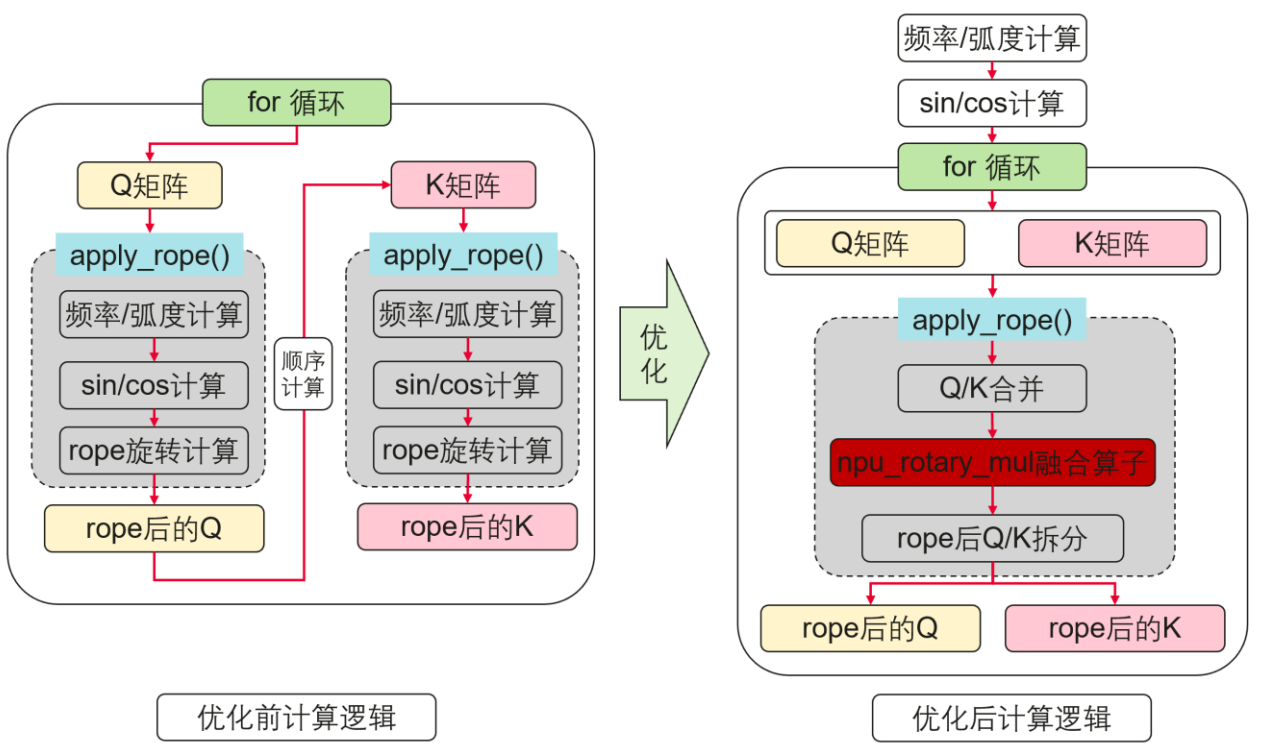
具身智能模型Pi0--昇腾适配及推理性能优化效果简介：

➢ Pi0原始开箱未优化推理性能，在昇腾A2服务器上单次推理为**670毫秒**

➢ 经过对Pi0的全方位昇腾优化适配，单次推理时间跑进**80毫秒**，

相比开箱性能提升830%。

优化策略1：旋转编码sin/cos计算逻辑优化



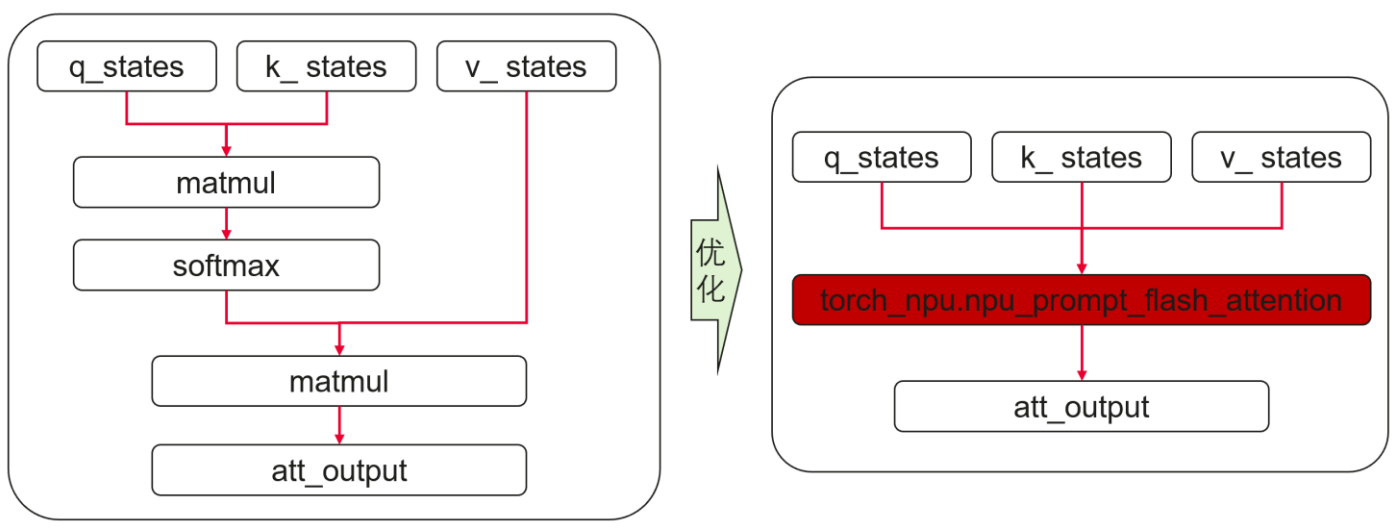
rope旋转编码优化示意图

```python
def apply_rope(query_states, key_states, cos, sin):
  n_q = query_states.shape[2]
  n_k = key_states.shape[2]

  merged_states = torch.cat([query_states, key_states], dim=2)

  merged_rot = torch_npu.npu_rotary_mul(merged_states, cos, sin)

  q_rot, k_rot = merged_rot.split([n_q, n_k], dim=2)

  return q_rot, k_rot
```

rope旋转编码优化关键代码片段

- **RoPE 模块(Rotary Position Embedding)作用**：解决传统位置编码长序列失效问题，精准捕捉 token 相对位置且支持长度外推；通过算子 / 硬件适配降低 RoPE 计算开销，减少推理时延、提升吞吐量。
- 通过修改paligemma_with_expert.py中class PaliGemmaWithExpertModel(PreTrainedModel)中forward函数中的旋转编码计算逻辑，修改apply_rope函数中的输入输出，利用**torch_npu.npu_rotary_mul融合算子，并将Q/K_states进行融合计算拆分**。
- 将sin/cos计算提到for循环外部进行统一计算，减少其重复计算次数等手段，可以实现旋转编码部分的优化加速。

优化策略2：flash attention的npu原生融合算子优化



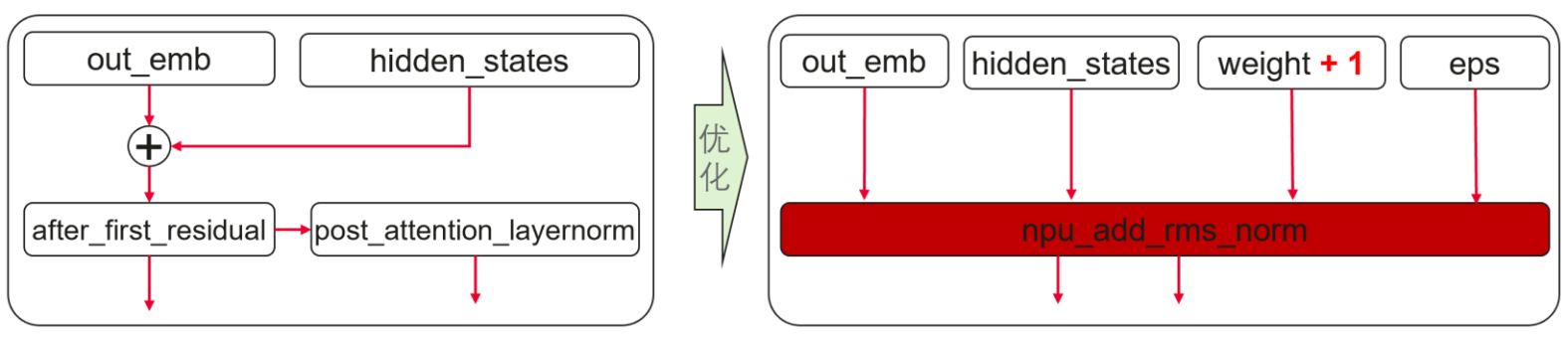
flash_attention部分
昇腾融合算子优化示意图

```python
attention_mask = torch.logical_not(attention_mask).to(dtype=torch.int8, memory_format=torch.contiguous_format)
attention_mask = attention_mask[:, None, :, :]

......

def eager_attention_forward(
    self, attention_mask, batch_size, head_dim, query_states, key_states, value_states
):
    att_output = torch_npu.npu_prompt_flash_attention(
        query_states,
        key_states.contiguous(),
        value_states.contiguous(),
        num_heads=self.num_att_heads_forward,
        input_layout="BSND",
        scale_value=self.scale_value,
        pre_tokens=65535,
        next_tokens=65535,
        atten_mask=attention_mask,
        num_key_value_heads=self.num_key_value_heads_forward
    )

    att_output = att_output.reshape(batch_size, -1, self.num_att_heads_forward * head_dim)

    return att_output
```

FA昇腾融合算子替换
关键代码片段

- **Flash Attention作用：** 通过分块 / 重计算将注意力显存占用从O(N*N)降至O(N)，解决长序列显存溢出；适配硬件访存特性提升计算速度、降低时延。
- 通过替换paligemma_with_expert.py中**FA部分为npu_prompt_flash_attention融合算子（目前优先使用npu_fused_infer_attention_score融合算子）**，复用融合算子在NPU上的快速计算能力，提升此模块的计算性能。同时，需要注意pi0中原始FA计算方式与npu_prompt_flash_attention融合算子在数学公式上的实现差别，尤其是在attention_mask上的数学公式实现差异(bool类型True位置转换为负无穷大），并进行相应的适配修改。
- 将FA中一部分**计算过程提到循环外面进行提前计算**，可以实现FA部分的优化加速。

优化策略3：layernorm部分npu原生算子优化



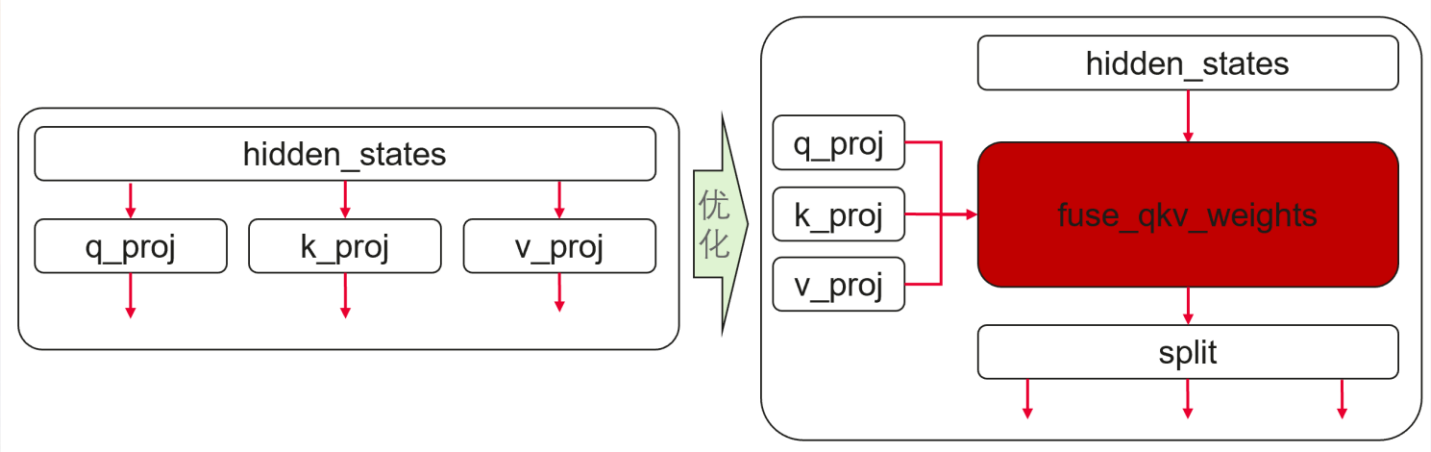
Layer norm部分
昇腾融合算子优化示意图

```python
hidden_states = torch_npu.npu_rms_norm(
    hidden_states,
    layer.input_layernorm.weight.add(self.ones_add),
    1e-6
)[0]

......

out_emb, _, after_first_residual = torch_npu.npu_add_rms_norm(
    layer.self_attn.o_proj(att_output[:, start:end]),
    hidden_states.to(torch.bfloat16),
    layer.post_attention_layernorm.weight.add(self.ones_add),
    1e-6
)
```

Layer norm昇腾融合算子替换
关键代码片段

- **LayerNorm 核心作用：**归一化每层输入特征，消除分布偏移以稳定训练 / 推理；通过可学习参数保留特征信息，增强模型特征捕捉能力。
- 将first residule计算中的**layernorm和add运算替换为NPU算子npu_add_rms_norm融合算子**，复用融合算子在npu上的快速计算能力，提升此模块的计算性能。同时，需要注意pi0中layernorm计算方式和npu_add_rms_norm融合算子在数学公式乘积系数上的实现差别，并进行相应的适配修改。
- 可以将paligemma_with_expert.py中class PaliGemmaWithExpertModel(PreTrainedModel)中forward函数**中原生的layer.input_layernorm计算部分用昇腾原生的npu_rms_norm进行替换**，可以实现layernorm部分的优化加速。

优化策略4：qkv计算部分npu融合计算优化



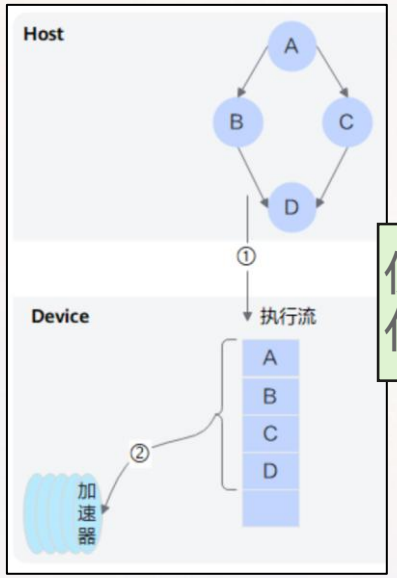
Q/K/V向量投影权重融合计算部分
昇腾融合算子优化示意图

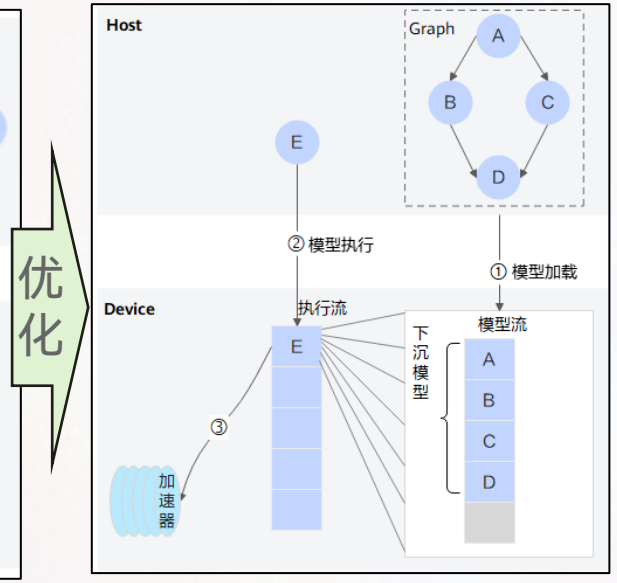
```python
# 将q/k/v权重融合为单个线性层qkv
@torch.no_grad()
def fuse_qkv_weights(self):
    """
    将每层 self_attn 的 q/k/v 权重 concat 成单个 qkv 线性层
    """
    for model in self.models:  # paligemma + gemma_expert
        for layer in model.layers:
            attn = layer.self_attn
            w_q = attn.q_proj.weight
            w_k = attn.k_proj.weight
            w_v = attn.v_proj.weight

            w_fused = torch.cat([w_q, w_k, w_v], dim=0).bfloat16()

            # 创建 qkv 线性层
            attn.qkv = nn.Linear(w_fused.shape[1], w_fused.shape[0], bias=False, device=w_q.device).to(torch.bfloat16)

            # 将拼接后的权重赋值给 qkv 权重
            attn.qkv.weight.data.copy_(w_fused)

......

qkv = layer.self_attn.qkv(hidden_states)

q_proj, k_proj, v_proj = qkv.split([q_out, kv_out, kv_out], dim=-1)
```

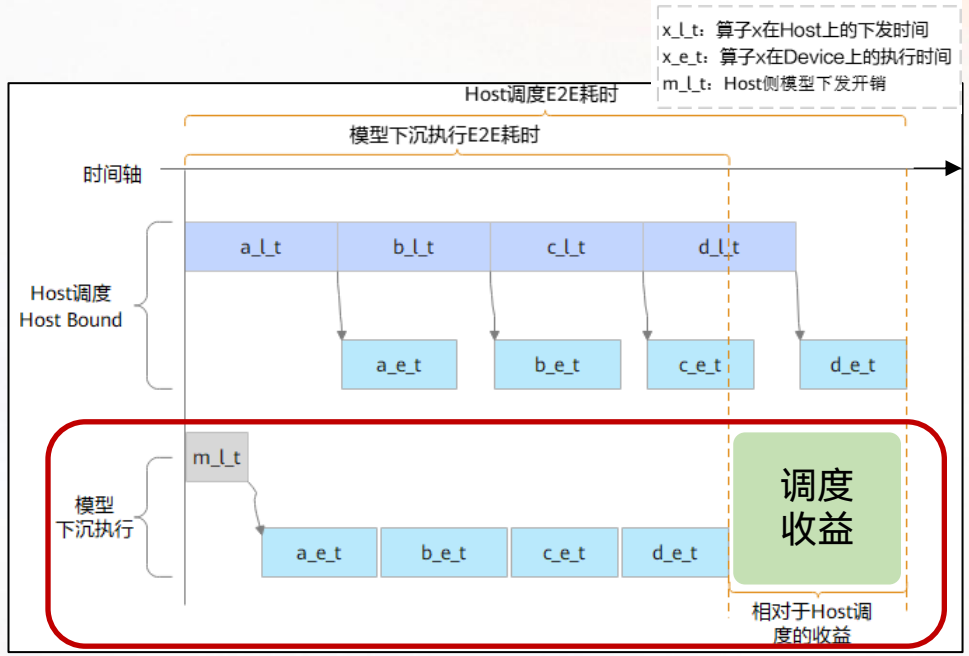
Q/K/V向量投影权重融合计算
关键代码片段

- **优化计算过程：**模型backbone层输入hidden_states通过投影矩阵线性转换为的Query / Key / Value向量计算的过程中。
- 将pi0模型前向计算过程中q/k/v的分离计算过程，转换为qkv融合权重进行计算，然后再对融合计算结果进行split切分，**节省缓存换入换出，用大矩阵乘法提升Cube的利用率**。通过对paligemma_with_expert.py中的q/k/v计算部分进行对应优化，可以实现qkv计算部分的优化加速。

优化策略5：图模式优化







原始模型HOST调度原理图

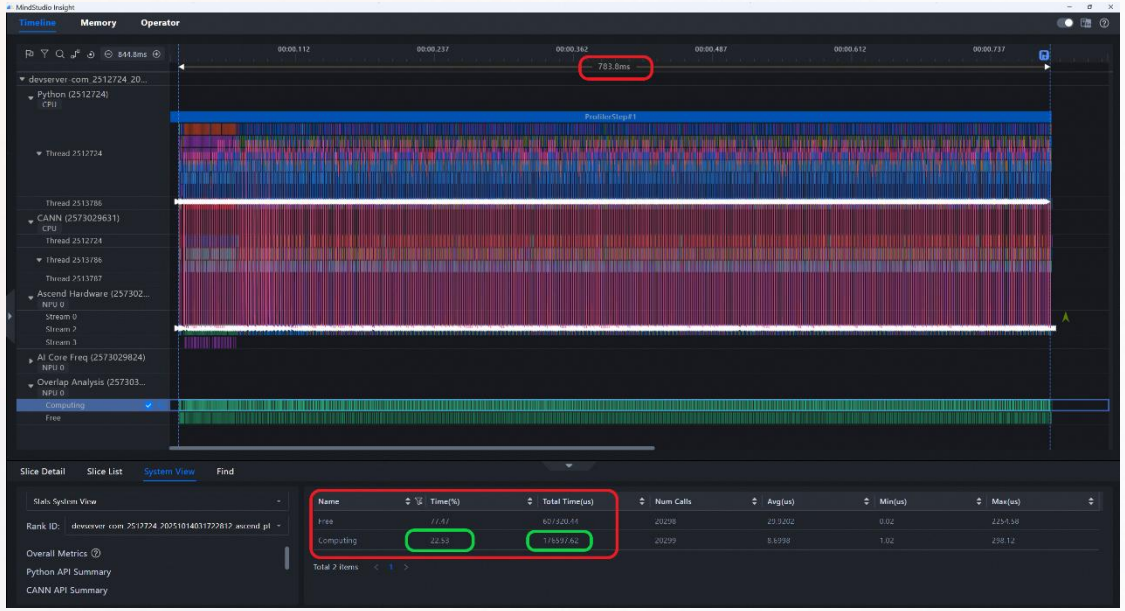
图模式模型下沉调度原理图：分为两个阶段，模型加载和模型执行

```python
import torch_npu
import torchair as tng
from torchair.configs.compiler_config import CompilerConfig

config = CompilerConfig()
config.experimental_config.frozen_parameter = True
config.experimental_config.tiling_schedule_optimize = True

npu_backend = tng.get_npu_backend(compiler_config=config)

......

self.compiled_embed_and_model_forward = torch.compile(
    self.embed_and_model_forward,
    dynamic=False,
    fullgraph=True,
    backend=npu_backend
)

self.denoise_step_all_compile = torch.compile(
    self.denoise_step_all,
    dynamic=False,
    fullgraph=True,
    backend=npu_backend
)
......

(
    prefix_pad_masks,
    dt,
    x_t,
    time,
    past_key_values
) = self.compiled_embed_and_model_forward(
    noise, images, img_masks, lang_tokens, lang_masks
)

for step in range(self.config.num_steps):
    self.denoise_step_all_compile(
        state,
        prefix_pad_masks,
        past_key_values,
        x_t,
        bsize,
        time,
        dt
    )
```

图模式下沉调度计算关键代码片段

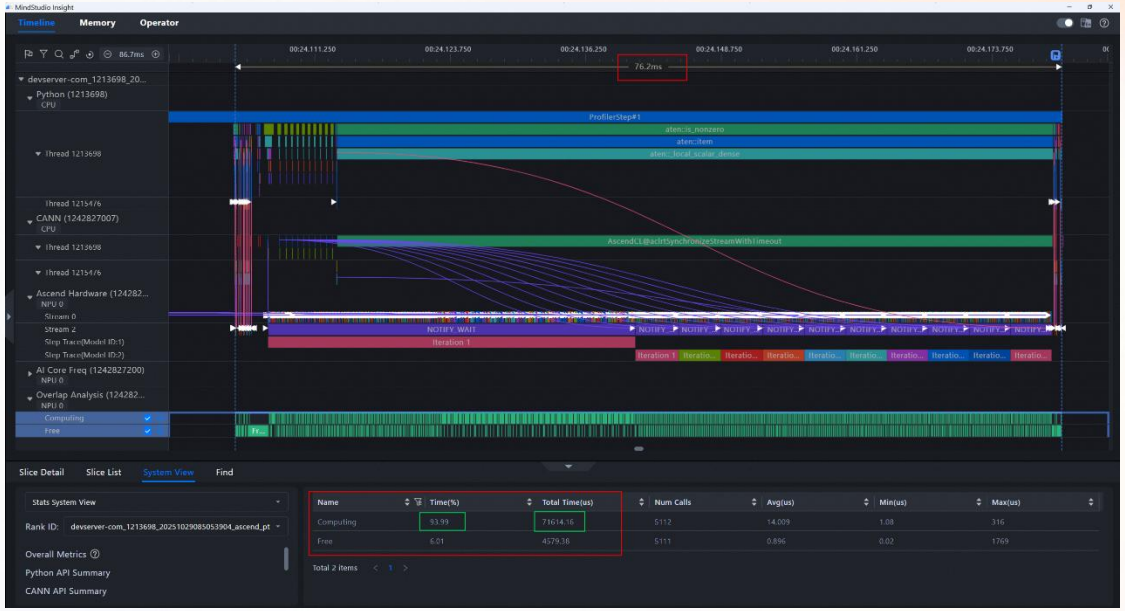
- 原生开箱未优化的 pi0 模型在昇腾 NPU 端推理时存在显著的 **host-bound 瓶颈**，CPU 与 NPU 间逐算子指令下发、同步等待的开销占比过高，导致设备侧算力无法充分释放。
- **基于torch_npu的torchair图模式编译能力，将pi0模型的快系统与慢系统推理逻辑整体下沉至NPU侧**，并通过整图编译的方式完成计算图构建与优化，大幅削减CPU-NPU跨设备交互频次，降低指令调度与同步延迟，最终实现推理全流程的性能加速。
- 因此，为了减少CPU到NPU的逐算子下发与同步开销，结合torch_npu的torchair图模式编译模块，可以在modeling_pi0.py中将**pi0的快系统与慢系统部分尽可能进行整图编译**，进而实现整体的计算优化加速。

5种优化策略协同作用



Pi0昇腾优化前开箱推理性能流水图

优化前后对比

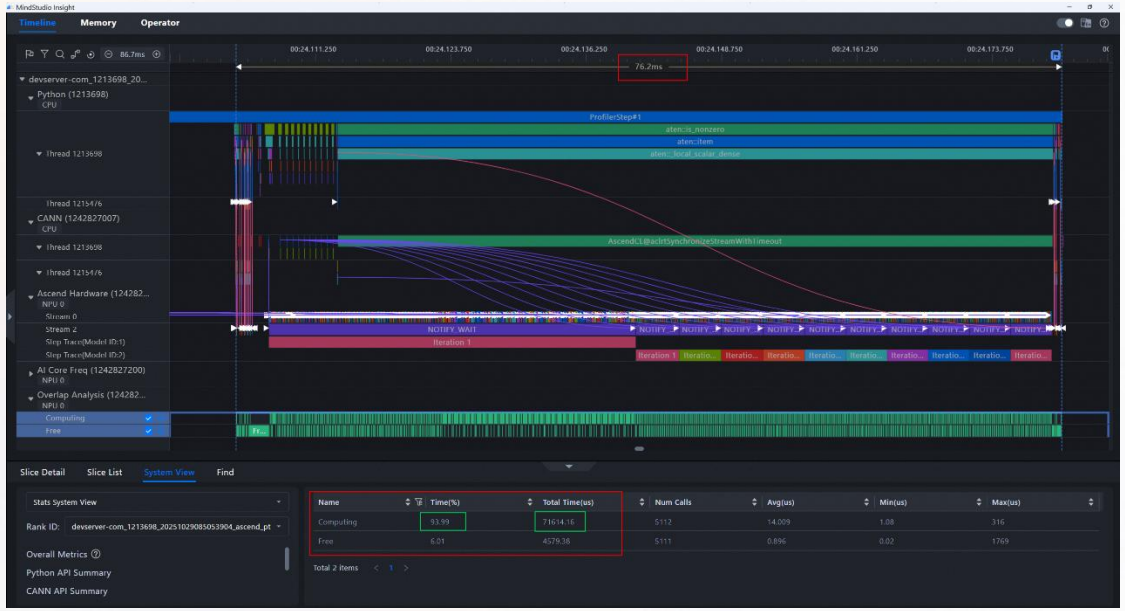


Pi0昇腾优化后推理性能流水图

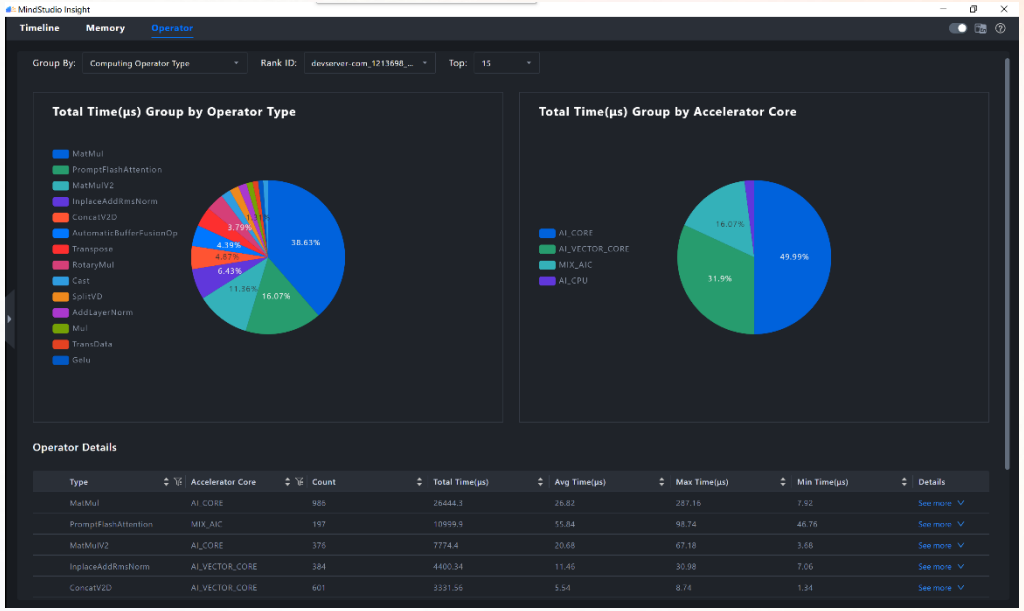
优化前后pi0昇腾A2推理MindStudio流水图的对比分析:

- **CPU到NPU的指令下发次数明显减少**。优化前的开箱流水图中，指令下发次数量级巨大，优化后只有简单的11条运算指令下发流水线（1条慢系统VLM，10条快系统action expert），极大地节省了通信开销，降低了NPU计算单元的空闲时间
- **推理时间明显减少**。从优化前端到端推理670毫秒降低到优化后的80毫秒，推理速度提升至开箱性能的830%。

5种优化策略协同作用



Pi0昇腾优化后推理性能流水图



Pi0昇腾优化后推理算子计算时间占比统计图

基于上述昇腾迁移-优化策略，对单次推理过程进行基于MindStudio Insight的profiling分析。其中，

- 昇腾npu计算时间总占比从**22.5%提升至94%**。
- 算子计算时间从**176.6毫秒降至71.6毫秒**。
- NPU空闲时间从607毫秒降至4.58毫秒。
- AI_CORE类型昇腾融合算子计算时间占比**从30%提升至50%**，FA算子占比**从1.2%提升至15.4%**。
- 由此可以得出，优化后的pi0推理过程，已经极大地降低了host-bound问题，也更好地利用了昇腾融合算子加速能力。

为进一步进行性能比较，避免profiling分析中的冷启动时延问题，进行100次连续推理计算时间取平均值的方式进行对比，得到**昇腾A2优化后的pi0推理时间为80ms，相比于昇腾开箱670ms推理时延性能提升至8.3倍左右**。

分析结果显示出迁移优化策略提升了pi0对昇腾的亲和程度，发挥了昇腾npu的融合算子优势和算力优势，并充分显示出具身智能pi0模型在**昇腾上部署的可行性及易用性**，为将来**CANN开源开放生态中pi0等机器人具身智能大模型在昇腾系列处理器上的训练-推理-部署**等提供了参考。

Pi0模型昇腾真机部署实验方案

05

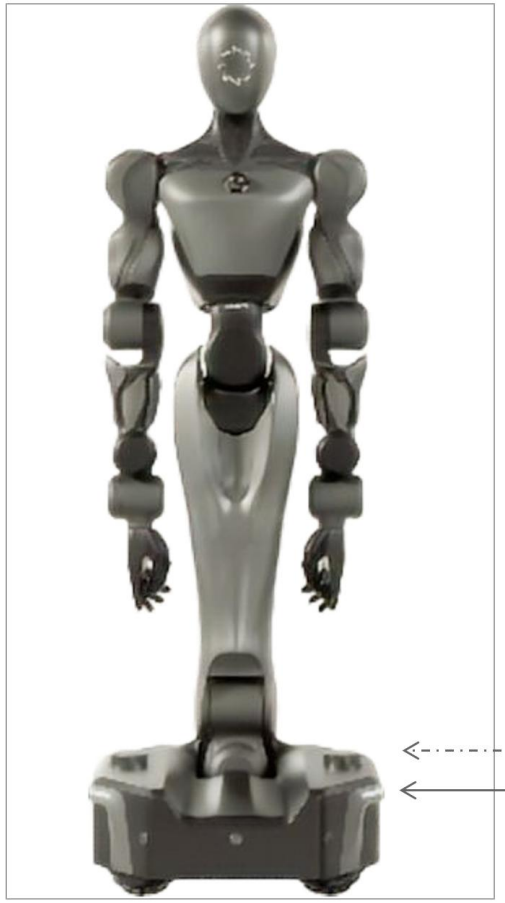
05 Pi0模型昇腾真机部署实验方案

➢ 如何基于昇腾软硬件真实地驱动机器人完成软硬件配置，如何连线通信

方案1：使用昇腾A2服务器进行pi0模型推理机器人动作，网线直连控制机器人运动

方案2：使用310P+Host小主机进行模型推理，并通过网线与机器人上位机通信。

(算力硬件与机器人解耦)



昇腾A2服务器+机器人推理搭建

A2服务器可**等效于**：Host小主机+310P

A2服务器

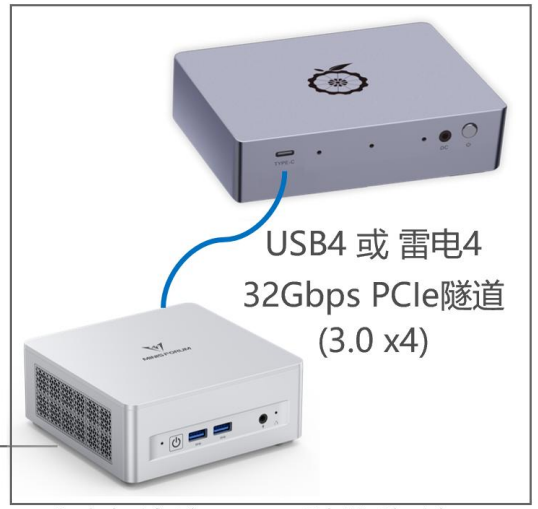
A2服务器
等效于
310P+迷你主机

昇腾A2服务器 连接 机器人上位机
(**网线**/光纤/WiFi **LAN**, 或5G公/专网)

网线连接机器人上位机
(同先前昇腾A2服务器方案)

图例：小主机连接外置显卡坞

昇腾替代



小主机连接 310P香橙派加速器

基于昇腾A2 / 310P平台的机器人具身智能VLA端到端大模型控制方案

05 Pi0模型昇腾真机部署实验结果

- 如何基于昇腾软硬件真实地驱动机器人完成操作任务
- 为具身智能领域提供成熟昇腾解决方案，欢迎更多合作伙伴开源共创

昇腾A2
+
机器人

昇腾+双臂机器人
桌面整理POC视频

昇腾310P
+
机器人

- 语言指令
- 双视角图像
- 机械臂关节状态

VLA模型推理

Pi0在昇腾的训推加速进一步优化方向

06

06 Pi0在昇腾的训推加速进一步优化方向

	其它优化方案简介
增量推理适配	基于固定KV Cache，构建pi0增量推理体系，针对机器人连续控制场景的增量输入(多帧图像、持续语言交互），复用历史计算结果，仅对新增特征片段做推理计算，进一步降低长时序任务的推理时延，适配实时控制需求。
KV Cache 优化	针对pi0模型推理过程中 KV 缓存动态更新带来的性能损耗，采用固定尺寸缓存方案，按机器人控制任务的序列长度上限预设缓存空间，避免动态内存申请与数据拷贝，提升缓存利用效率。
多卡并行训推加速优化	利用昇腾HCCL高效集合通信，采用数据并行与混合并行等多卡并发调度及负载均衡手段，为具身大模型提供多卡分布式训练与多流并行推理一体化加速方案，解决“训练在GPU、推理在NPU”的跨平台精度对齐问题，提升机器人控制、视频理解与时序决策的吞吐与实时性，整体加速长时序高算力任务。
端到端全链路优化	将模型前后处理环节(图像解码、归一化、关节角度序列格式化）迁移至NPU执行，通过NPU原生图像处理算子替换CPU计算，消除数据传输延迟，进一步降低空闲时间，目标将推理总耗时压缩至更低。
工具链与硬件兼容性提升	适配业界主流的大模型训推框架及昇腾工具链，优化代码适配逻辑，增强跨硬件型号的兼容性，同时推动优化策略的模块化封装，形成可一键调用的昇腾优化工具包，降低其他模型的迁移成本。

cann-recipes昇腾具身智能开源仓介绍

07

07 cann-recipes昇腾具身智能开源仓

扫码进入昇腾CANN具身智能仓库地址

- 上述pi0适配昇腾平台并使能性能优化的相关代码，还有对应的昇腾环境搭建说明已在**gitcode平台cann/cann-recipes-embodied-intelligence仓开源**，可以按照pi0昇腾迁移优化README文档进行使用。目前该代码仓也已经开源了**pi0 / pi0.5 / ACT / openVLA / Gr00t-N1.6 / Alpamayo-R1 / Cosmos-predict 等具身智能机器人VLA大模型、自动驾驶大模型、世界模型在昇腾平台的训推代码**。

扫码

✨样例列表

操作类模型 (Manipulation)

实践	平台	框架	简介	性能参考
Pi0	Atlas A2	torch	基于LeRobot库，在Atlas A2环境适配Pi0模型，通过使能融合算子、图模式、计算逻辑优化等手段，实现了较低的推理时延。	**80 ms**
Pi0	Ascend 310P	OM	基于LeRobot库，在310P环境OM离线推理的Pi0模型，实现了较低的推理时延。	**~270 ms** (OrangePi AI Station)
Pi0.5	Ascend 310P	torch	在310P环境基于PyTorch直接进行Pi0.5模型在线推理。	**~862 ms**
Pi0.5	Ascend 310P	OM	在310P环境OM离线推理的Pi0.5模型，实现了较低的推理时延。	**~410 ms**
Pi0.5	Atlas A2	torch	在Atlas A2环境进行Pi0.5模型的训练，精度正常，性能达到较优的水平。	**69.19 samples/s** (优化后)
ACT	Ascend 310P	OM	在310P环境OM离线推理的ACT模型，实现了较低的推理时延。	**~200 ms** (OrangePi AI Station)
DiffusionPolicy	Ascend 310P	OM	在310P环境OM离线推理的DiffusionPolicy模型，实现了较低的推理时延。	-
OpenVLA	Ascend 310P	OM	在310P环境OM离线推理的OpenVLA 7B模型，实现了较低的推理时延。	-
Isaac-GR00T N1.6	Atlas A3	torch	通用人形机器人基础模型，适配昇腾A3平台。	-
Spirit v1.5	Ascend 310P	torch	千寻智能自研的具身智能模型，RoboChallenge评测综合排名第一(截至2026.1.12)。	-

世界模型 (World Model)

实践	平台	框架	简介	性能参考
Cosmos-Predict2.5-2B	Atlas A3	torch	Cosmos世界基础模型，支持文本/图像生成世界(Text2World/Image2World)，生成物理一致的视频。	**~920 s** (生成5.8s视频)
Cosmos-Transfer2.5-2B	Atlas A3	torch	Cosmos世界基础模型，支持多控制信号(深度图/语义分割/边缘检测等)的视频风格转换。	-

导航模型 (Navigation)

实践	平台	框架	简介	性能参考
Alpamayo-R1	Atlas A2	torch	面向L4/L5级智能驾驶的VLA大模型(10B)，支持因果思维链推理。	**~7.32 s** (生成10条预测轨迹)

运动控制模型 (Locomotion)

实践	平台	框架	简介	性能参考
LQC	Atlas A2	torch	足式机器人的强化学习运动控制器，适用于G1、GO2等主流机器人型号。	-

昇腾CANN开源开放-具身智能仓库地址

https://gitcode.com/cann/cann-recipes-embodied-intelligence

欢迎大家加入昇腾具身智能开源共建

欢迎大家加入右侧CANN-recipes交流群
同时扫描下方具身、大模型推理仓库二维码
贡献您的优秀昇腾适配案例

扫码

扫码

群聊：cann-recipes交流群

该二维码7天内(4月24日前)有效，重新进入将更新

➢ 昇腾CANN开源开放-具身智能仓库地址
https://gitcode.com/cann/cann-recipes-embodied-intelligence

➢ 昇腾CANN开源开放-大模型推理仓库地址
https://gitcode.com/cann/cann-recipes-infer

Thanks!

访问CANN开源社区

关注昇腾CANN公众号